॥ श्रीगणेशाय नमः ॥

Informed Search

- These search algos use some domain-specific hints about the location of the goal(s).
- Can find solutions more efficiently than uninformed searches.
- These hints are given as a function:

$h(n)$ = estimated cost of the cheapest path from the state at node n to a goal state.

Heuristics

- Using our knowledge of the problem to consider some successors of the current state.
- Pruning the state space for gaining speed, but perhaps missing the correct solution.
- We must be able to rank the successors of a node.

n → heuristic func^n → estimated cost to the goal of all successors

E.g. Euclidean distance

Best first search

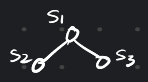
- Priority queue
- Using an evaluation function to decide which adjacent node is the most promising and then explore it.

```
BEST_FIRST
q = priority_queue()
q.enqueue(start, 0)
while q is not empty:
    t = q.dequeue()
    if t == goal:
        return t
    else:
        for t' in expand(t, cost_function):
            if (t' is not visited) or (t'.cost < visited[t'].cost):
                mark t' as visited
                q.enqueue(t')
```

```
expand(node n, cost_function cf):
    nodes = []
    for n' in neighbors(n):
        n'.cost = cost_function(n, n')
        nodes.append(n')
    return nodes
```

BeFS → BFS, UCS, greedy best-first, A*

If cost func^n = depth(n) + 1 → BeFS = BFS



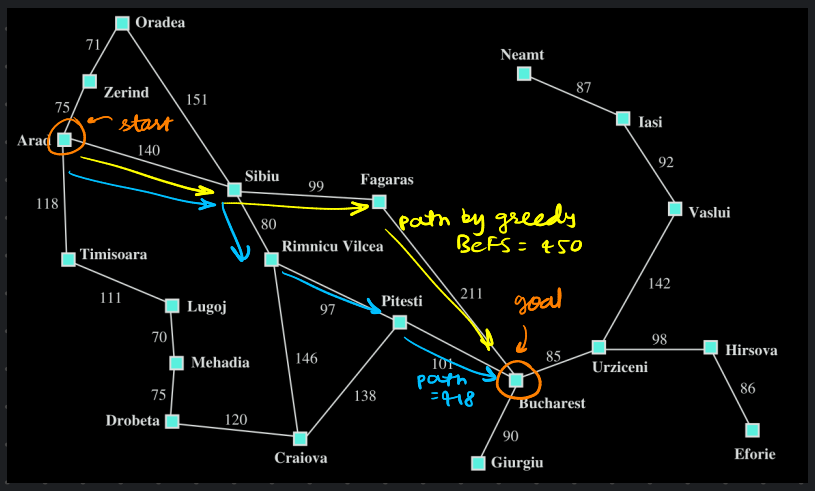
$(S1, 0), (S_2, 1), (S_3, 1)$

If cost func^n = {min cost from start to n} + {next node's cost} → UCS

Greedy best-first search

- This search algo will expand the node that appears to be the closest to the goal.
- Evaluation function:

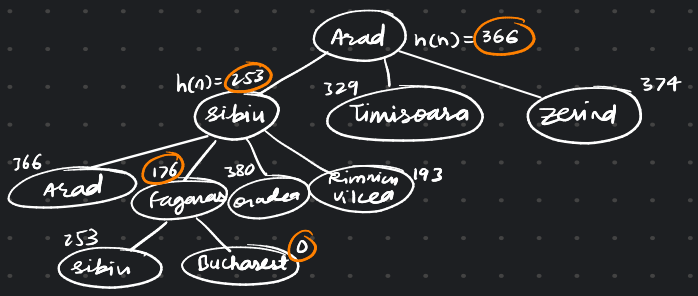
$$f(n) = h(n)$$



straight-line distance is used as the heuristic for route-finding problems.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.16 Values of h_{SLD}—straight-line distances to Bucharest.



Total cost = 140 + 99 + 211 = 450
Not the optimal soln.

Performance analysis

Complete? No. In finite search spaces, it is complete.
Also might end up in loops.
Due to greediness, we might prune solutions.

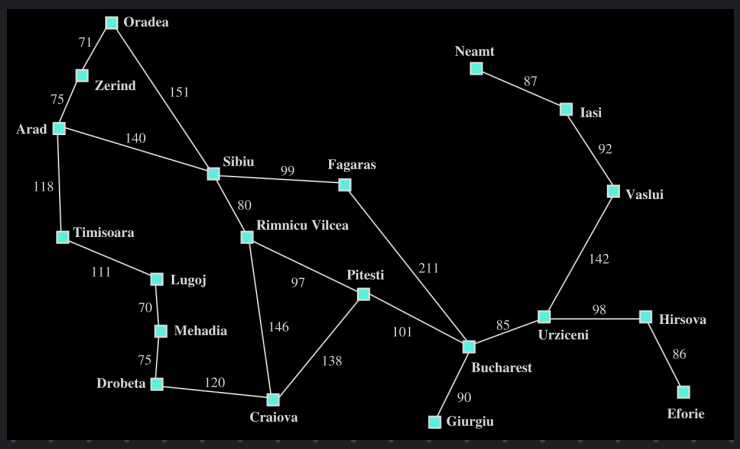
Time? $O(b^m)$ ← generally, much faster. $O(bm)$ for certain problems.
Improves if a good heuristic is used.

Space? $O(b^m)$

Optimal? No.

A* search $f(n) = g(n) + h(n)$

- Idea: avoid expanding paths that are already expensive
- Cost function: $f(n) = g(n) + h(n)$
- A* is just Dijkstra's with a sense of direction.

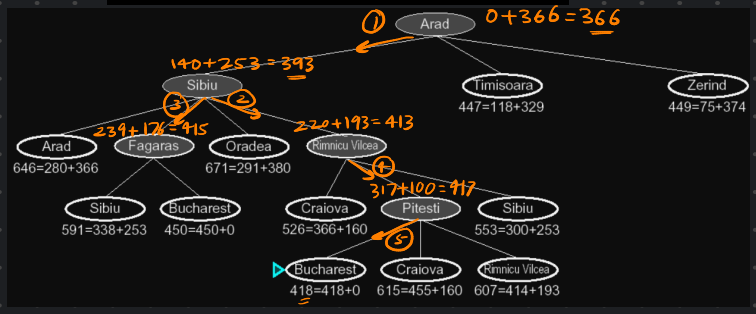


$g(n)$ → cost so far to reach n

$h(n)$ → estimated cost from n to goal

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.16 Values of h_{SLD}—straight-line distances to Bucharest.



Goal test:

- In best first search, goal test: if (t == g) return t
- In A*: if (t == goal) and ($f(t) \leq f(q.top())$) return t
  ensures optimality

queue at first: Sibiu = 393, Timisoara = 447, Zerind = 449 ①

queue: (Timisoara, 447), (Zerind, 449), (Arad, 646), (Fagaras, 415), (Oradea, 671), (Rimnicu V., 413) ②

queue: (Timisoara, 447), (Zerind, 449), (Arad, 646), (Fagaras, 415), (Oradea, 671), (Craiova, 526), (Pitesti, 417), (Sibiu, 553) ③

queue: (Timisoara, 447), (Zerind, 449), (Arad, 646), (Oradea, 671), (Craiova, 526), (Pitesti, 417), (Sibiu, 553), (Sibiu, 591), (Bucharest, 450) ④

We don't choose goal as we have a smaller option

queue: (Timisoara, 447), (Zerind, 449), (Arad, 646), (Oradea, 671), (Craiova, 526), (Sibiu, 553), (Sibiu, 591), (Bucharest, 450), (Bucharest, 418), (Craiova, 615), (Rimnicu V., 607) ⑤

(goal)

Admissible heuristics

An admissible heuristic is one that never overestimates the cost to reach the goal state.

$$\forall n,\ h(n) \leq h^*(n)$$

$h^*(n)$ → ideal heuristic

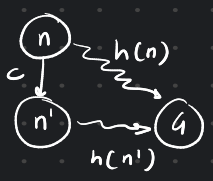
$h^*(n)$ = value of optimal cost from node n to the goal.

An admissible heuristic is optimistic.
With an admissible heuristic, A* is optimal.

If $\exists n,\ h(n) \geq h^*(n)$ → h is not admissible. → can't comment on optimality.

E.g. SLD → Euclidean distance,
manhattan distance (L1 norm)] → when are moving on a grid

Consistent heuristics

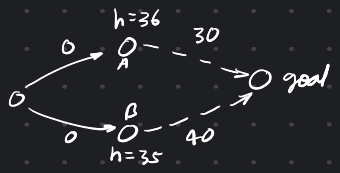


$\forall n$, $\forall$ successors n'

$$h(n) \leq c + h'(n)$$

NOTE: every consistent heuristic is also admissible.

Q. Assuming that a perfect heuristic function, h(n) = h*(n), doesn't exist, there are only 2 possibilities:
1. It underestimates the distance to the goal.
2. It overestimates the distance to the goal.
Which is better?



If h underestimates:
$f(A) = 36$
$f(B) = 35$ ← we'd explore this.
Then we'd get (g, 40) and (A, 36) so we won't opt for g before exploring A.

If overestimation would've been allowed,
say $h(A) = 1{,}000$.
In queue: (g, 40), (A, 1000)
we'd directly go for (g, 40) since our heuristic tells us that A is at a greater distance.

So a heuristic function that underestimates is better.

Proof of optimality of A*

(proof by contradiction)

Suppose there's an optimal path with cost C^*, but the algo (A^*) returns a path with $C > C^*$.
Assume that there's a node n in the optimal path which remains unexplored.



In the order in which nodes are arranged in the priority queue is the f(n) values.



$f(n) > C^*$

If (n, f(n)) is present after (goal, C^*), it won't be explored.

For any node n,

$$f(n) = g(n) + h(n)$$
$$= g^*(n) + h(n)$$

$g(n) = g^*(n)$ as n lies on the optimal path

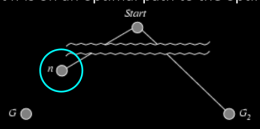
$$h(n) \leq h^*(n)$$
$$\Rightarrow g^*(n) + h(n) \leq g^*(n) + h^*(n)$$
$$\Rightarrow f(n) \leq g^*(n) + h^*(n)$$
$$\Rightarrow f(n) \leq f^*(n)$$
$$\Rightarrow f(n) \leq C^*$$

↳ contradiction. The assump^n that the algo will return a suboptimal soln is incorrect.

$\Rightarrow$ A*, with an admissible heuristic, will give you an optimal solution.

Proof of Optimality of (Tree)A*

- Assume *h*() is admissible.
  Say some sub-optimal goal state G_2 has been generated and is on the frontier.
  Let *n* be an unexpanded state such that *n* is on an optimal path to the optimal goal *G*.



$f(G_2) = g(G_2)$	since $h(G_2) = 0$
$g(G_2) > g(G)$	since G_2 is suboptimal
$f(G) = g(G)$	since $h(G) = 0$
$f(G_2) > f(G)$	substitution

Now focus on n:

$h(n) \leq h^*(n)$	since h is admissible
$g(n) + h(n) \leq g(n) + h^*(n)$	algebra
$f(n) = g(n) + h(n)$	definition
$f(G) = g(n) + h^*(n)$	by assumption
$f(n) \leq f(G)$	substitution

Hence $f(G_2) > f(n)$, and A* will never select G_2 for expansion.

Complete? Yes.

Time? $O(b^d)$

worst case: say $h(n) = 0$.
A* search will work as UCS.

Actually should be given in terms of branching factor b^* since a good heuristic prunes off branches.
b^*, ideally, should be zero.

Space? $O(b^d)$

Optimal? Yes, if an admissible heuristic is used.

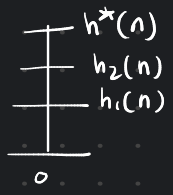
More informed heuristic means less search.

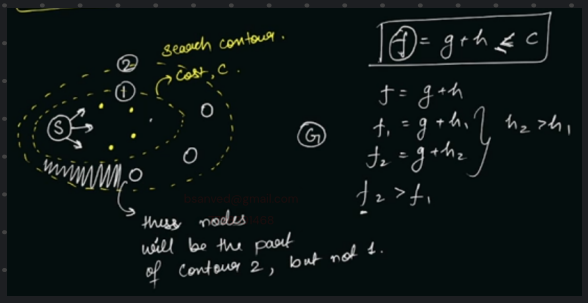
Let h_1 and h_2 be admissible.

$\forall n,\ h_2(n) \geq h_1(n)$

↳ h_2 dominates h_1

then $\forall n\ (\text{expands}(A_2, n) \supseteq \text{expands}(A_1, n))$





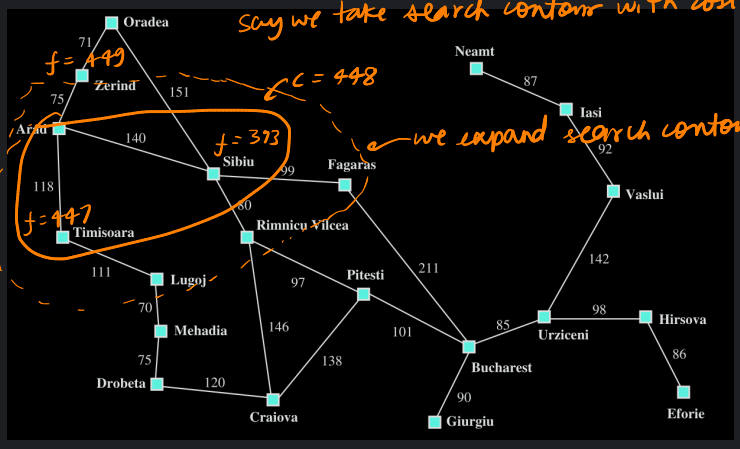
Say $h_2 > h_1$, then h_2 is closer to h^*.

h_2 will be more accurate than h_1 in providing the remaining estimated cost.

(anyways they won't overestimate, so higher value means closer to C^*).

h_2 is more informative than h_1.

h_2 will always dominate h_1.



Say we take search contour with cost 448

$f = 449$

$C = 448$

$f = 393$

$f = 447$

we expand search contour with each step.

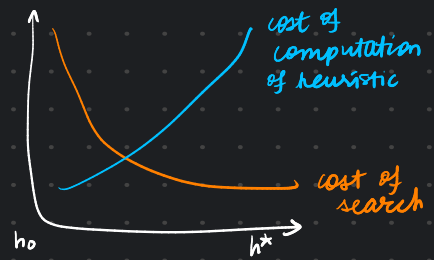
$f = g + h$

Say $h_2 > h_1$

(more informed) good heuristic → less search

using a bad heuristic → good or bad for search? bad for search → more search

using a good heuristic → good or bad for search? bad for search → less searches, but finding a good heuristic adds to computaional cost...



Memory bound A*

We need to solve the exponential memory problem of A*. We could introduce a DFS-like behavior.

Beam search

- Limit the size of the frontier (priority queue).
- Choose a size k and keep only k nodes with the best f values.

Drawback:

- Suboptimality and incompleteness

Iterative deepening A*

- Here we'll limit the value of f (we limited the value of depth in iterative deepening DFS).

```
limit = ϵ
while (q is not empty):
    DFS with pruning, remove n, (f(n) > limit)
    limit += ϵ
```

Complete? Yes.

Time? $O(b^d)$

Space? $O(bd)$

Optimal? Yes, if an admissible heuristic is used.

Weighted A*

The pull and the weighted push.

$$f(n) = g(n) + w \cdot h(n) \; ; \quad w \geq 1$$

$w = 1 \Rightarrow$ regular A*

$w = 0 \Rightarrow$ uniform cost search

$w \to \infty \Rightarrow$ greedy best first search

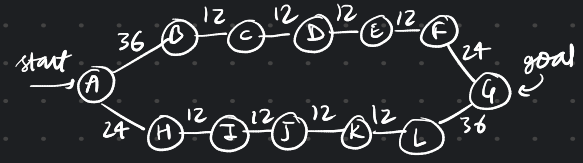
$1 < w < \infty \rightarrow$ weighted A*

In weighted A*, we can shrink the search contours further, but we'll lose admissibility.
Fewer states are explored, so if the optimal path ever strays outside the weighted search's contour, then the optimal path is missed.
If the optimal cost of A* is c*, then for weighted A*,

cost of solution, $C_w \in [C^*, w \cdot C^*]$ ← sub-optimal cost

Indirectly, we change h to wh. So admissibility may not hold. $w \cdot h(n)$ can be $> h^*(n)$.

Q.

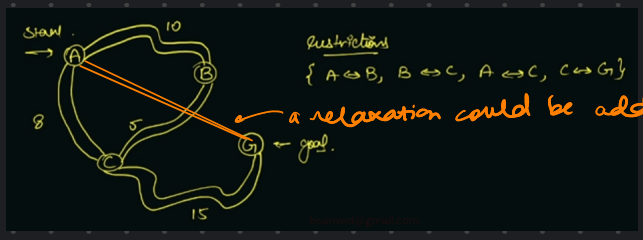


n	h(n)	n	h(n)
A	70	G	0
B	60	H	70
C	50	I	60
D	40	J	50
E	30	K	40
F	20	L	30

Find the path if
① A* is applied.
② weighted A* (w=2) is applied.

Relaxed problems

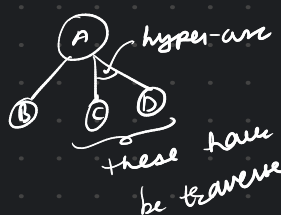
In every search problem, we have a set of rules that govern the actions we can take to make the next move.
Relaxing these rules, removing some restrictions, leads to a relaxed problem.



The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem.

AO* search

AND/OR graphs :



$A \rightarrow B$
or $A \rightarrow (C \& D)$

The nodes in the search space have a heuristic value, that is an estimate of the cost of solving the node.

At any point, AO* maintains a graph generated so far.

AO* is an extension of A* in which AO* is adapted to handle the dual-structure of decision making.

AO* search

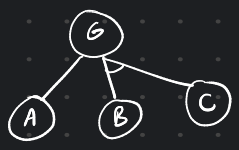
Steps :

1. Initialization : Start the search with the root node.
2. Node selection: The node with lowest cost is selected.

$$f(n) = g(n) + h(n)$$

either 1 of the OR nodes or one of the AND nodes based on the min. cost

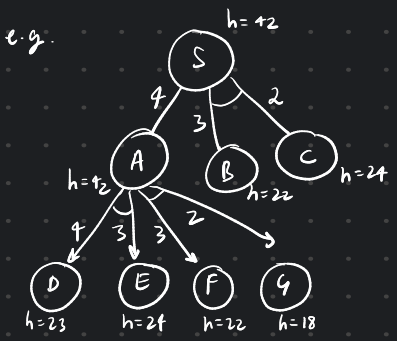
3. Node expansion : Generating the successors of the selected node and calculating the cost for each.
4. Back propogation : If a better (low-cost) path is found in the search, the algo updates the parent nodes and continues the propogan in the backward direction.
5. Termination :

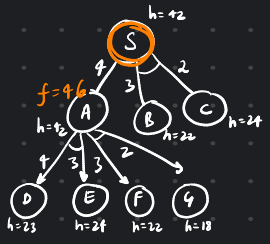


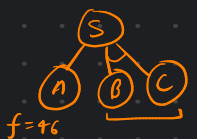
we find a solution in either the AND path or the OR path.
i.e. when a valid soln is formed.
(all nodes in any hyperarc are resolved)

The search stops when a valid soln is found.
(not necessarily optimal)

e.g.

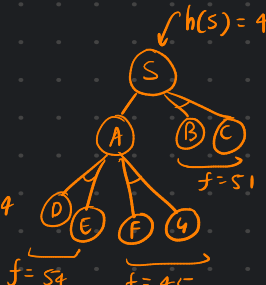




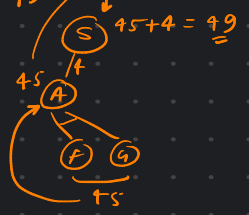


$f = 3+22+2+24 = 51$

Now choose A and back-propogate

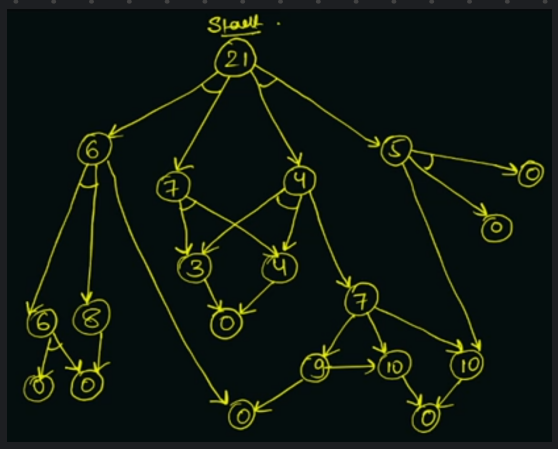


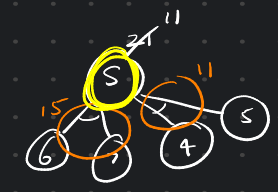
choose (F&G)

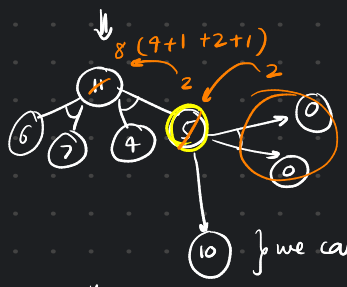


we are done with the search.
Backpropogate to A → Back prop. to S.

consider h values as given and edge costs as 1.

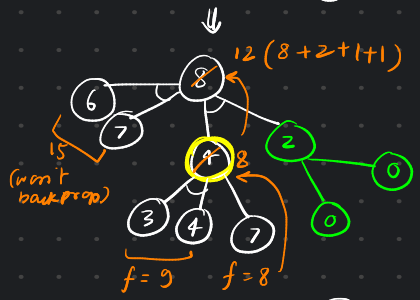




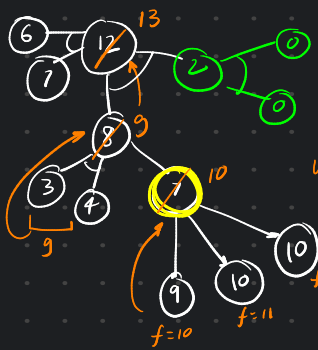


5 is terminated now. values are backed up so we can ignore it.

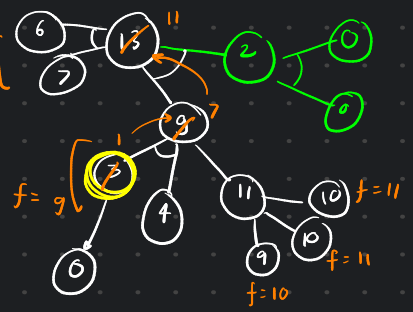
we can safely ignore this now



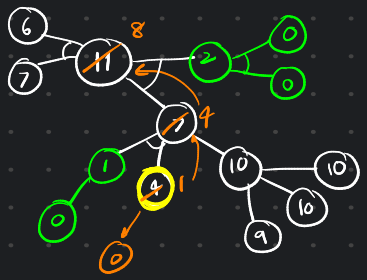
(won't backprop)

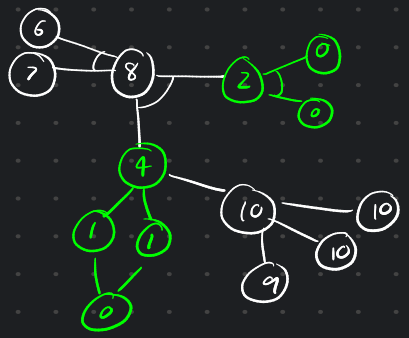


we have choice to backprop 9 or 11 (10+1) so we backprop 9.



3 is terminated now





so S is terminated as well. cost of search = 8

NOTE: heuristics are over-estimating. can NOT guarantee the optimal path.

sub-optimal path.

There's no concept of admissibility in AO* search.